

Formatter specializations for the standard library

Document #: P2930R0
Date: 2023-07-15
Project: Programming Language C++
Audience: LEWG
Reply-to: Mark de Wever
<koraq@xs4all.nl>

Contents

- [1 Introduction](#)
- [2 Motivation](#)
- [3 Scope](#)
 - [3.1 Atomic values](#)
 - [3.2 Random engines and distributions](#)
 - [3.3 Pointers and smart pointers](#)
 - [3.4 Utility types](#)
 - [3.5 `filesystem::path`](#)
 - [3.6 `mdspan`](#)
 - [3.7 `flat\_map` and `flat\_set`](#)
 - [3.8 Diagnostics library](#)
 - [3.9 Other types](#)
 - [3.10 `Byte`](#)
 - [3.11 Summary](#)
- [4 Design decisions](#)
 - [4.1 Format's impact on translation unit size](#)
 - [4.2 `Complex`](#)
 - [4.3 `Bitset`](#)
 - [4.3.1 `Bitset` reference](#)
 - [4.4 `Sub\_match`](#)

- 4.5 Diagnostics
 - 4.5.1 To specialize `wchar_t` or not to specialize `wchar_t`
 - 4.5.2 `Error_code`
 - 4.5.3 `Error_condition`
 - 4.5.4 `Error_category`
- 4.6 Byte
- 5 Impact on the standard
- 6 Implementation experience
- 7 Proposed wording
 - 7.1 Feature test macro
 - 7.2 Format’s impact on translation unit size
 - 7.3 Formatter complex
 - 7.4 Formatters `bitset` and `bitset::reference`
 - 7.5 Formatter `sub_match`
 - 7.6 Header `system_error`
 - 7.7 Byte
- 8 References

§ 1 Introduction

C++20 introduced `std::format` and in C++23 several improvements were made, for example [P2286R8] and [P2693R1]. Some types in the standard library are formattable that do not have streaming support, unfortunately the opposite is also true. Some types can be streamed, but not formatted. This paper looks at the types that are not formattable and proposes formatter support for some of these types. Other types will be out of the scope of this paper, but are identified.

§ 2 Motivation

The addition of `std::format` and `std::print` are a huge step forward for formatting and printing objects in C++. Unfortunately, for some types in the standard library, users still need to use the stream operators to format the output. A part of these types have been identified in [P1636R2]. Unfortunately the author of the paper can no longer be reached. (The author of this paper and others have, without success, reached out to the author of [P1636R2].) Parts of the design of the paper are approved by LEWG, but never made it into the standard.

This paper has a different approach for the formatter specializations as [P1636R2]. That paper was based on the state of <format> in C++20. In C++23 <format> gained a lot of new features giving better tools to define these specializations.

§ 3 Scope

The proposal [P1636R2] has identified some formatters that are missing. This proposal looks at more types.

§ 3.1 Atomic values

These types are not streamable nor formattable. This paper does not propose to add these types. Users can already format the values by do this by using

```
std::format("{} ", atomic_value.load());
```

This allows to specify the memory ordering of `load()`. Adding a formatter for this type would need to have a way to specify the memory order too. This adds extra complexity to the standard for little gain. The only benefit from having it in the library would be that it's possible to format a `vector<atomic<T>>` directly.

Users can already do this by using something like:

```
std::string example() {
    std::array input{
        std::atomic<int>{1},
        std::atomic<int>{2},
        std::atomic<int>{3}
    };

    return std::format("{} ",
        input | std::views::transform([](auto& i) { return i.load(); }));
}
```

§ 3.2 Random engines and distributions

These types are streamable to store and load their internal state. This is a set of decimal numbers, so it could be formatted as a range of integrals. However the intention is recreating a random engine or distribution with a previous state. It's not intended to display the state. The author sees little benefit from making these types formattable so the paper does not propose to make these types formattable.

§ 3.3 Pointers and smart pointers

Currently only void pointers and `nullptr`s can be formatted. It might be useful to format other pointers and smart pointers as well. [P1636R2] proposed this, but LEWG requested the smart pointers to be removed. Formatting pointers sounds useful. However there are

several design decision to be taken. The author feels this would better be addressed in a separate paper.

§ 3.4 Utility types

There are several utility types: `optional`, `variant`, `any`, and `expected` that would be nice to format. [P2286R8] introduced formatting for `tuple` and `pair`, but didn't offer a way to specify the formatting of the underlying types. Earlier versions of the paper proposed a solution, but there were issues with that approach. Later revisions removed this part of the proposal. These utility types would have the same issue. The author feels it would be better to solve this issue before adding more formatters with the same issue.

§ 3.5 `filesystem::path`

This was proposed in [P1636R2] and Victor Zverovich recently proposed [P2845R0].

§ 3.6 `mdspan`

This type has no formatting support. It would be useful to be able to format this type. However determining the best approach how to format this type has several interesting non-trivial design choices. Especially for `mdspans` with more than two dimensions. It seems better to do this in a dedicated paper. Adding a formatter for `mdarray` ([P1684R5] not in the WP yet) has been requested by LEWG. That formatter can be used for `mdspan` too.

§ 3.7 `flat_map` and `flat_set`

There is no need to do anything for these container adaptors. These types are handled by [P2585R0] and are formatted the same way as their “non-flat” equivalents.

§ 3.8 Diagnostics library

In the diagnostics library `error_code` is streamable but not formattable. The similar type `error_condition` is neither streamable nor formattable. The `stream` operator of `error_code` uses the value of the `error_category` which is not streamable. [P1636R2] proposes formatting of `error_code`.

It seems useful to make all these types formattable, especially the difference between `error_code` and `error_condition` looks odd. The author has not been able to find the historic reason for this discrepancy.

§ 3.9 Other types

There are some other types proposed in [P1636R2] which have not been incorporated in the standard library:

- `bitset`
- `complex`
- `sub_match`

LEWG was happy to add formatters for these types so this proposal adds them too.

§ 3.10 Byte

There are no proposals to add this type. Before [P2286R8] a byte formatter would only be able to format one element. After [P2286R8] it is possible to format ranges of elements. Since byte is intended to be used to give a byte-oriented access to a memory range this seems very useful to format. This makes it easy to print data received in buffers. For example for logging or debugging data received.

§ 3.11 Summary

There are several standard library types that are interesting to be formattable, that are not in the current standard. This paper proposes formatters for the following types:

- `bitset`
- `byte`
- `complex`
- `error_category`
- `error_code`
- `error_condition`
- `sub_match`

This paper does not propose formatters for the following types:

- (smart) pointers
- utility types (`optional`, `variant`, `any`, and `expected`)
- filesystem path
- `mdspan` (and `mdarray`)

§ 4 Design decisions

The proposal [P1636R2] is quite old, since that time the formatting library has improved and other formatter specializations have been added. For example, the paper [P2286R8] solved some of the same problems [P1636R2] solves for `complex`. So it's better to follow the design already in the Standard instead of the original design. This improves the consistency of the formatter specializations.

From an implementers point of view some of the design choices are not optimal. Forcing public inheriting from `formatter<basic_string<charT>, charT>` is not always optimal. For example, when a `sub_match` has a `contiguous_iterator` there is no reason to copy the matched result into a `basic_string`, using `basic_string_view` would avoid the copy. The paper [P2693R1] which proposed a part of [P1636R2] went used a different direction; only specifying what should happen without forcing design choices on implementers. This paper will follow the same approach; giving implementers the freedom to use the best option for their implementation.

§ 4.1 Format's impact on translation unit size

Per 22.14.6.3 [format.formatter.spec]/2

```
Each header that declares the template formatter provides the following
enabled specializations:
```

```
...
```

This requires every header that specializes a formatter to make the listed specializations available. This seemed fine when formatters were used in a limited number of headers, however since its initial inception formatters have been added to more headers in the standard library. This proposal adds more formatter specializations, causing more headers to be affected.

The `libc++` standard library uses granularized headers to reduce the size of translation units. This formatter requirement requires `libc++` to add extra includes in its implementation. Instead this paper proposes to only require these specializations to be available when the header `format` is included. This header defines the format functions that use these specializations.

The size increase of headers due to the inclusion of `format` has an impact on users. For example, one of the people working on Chromium mentioned the following in a [Phabricator](#) review

```
In Chromium we noticed that this almost doubled the preprocessed size of
<vector>, from ca 1.6 MB to 3.2 MB. Since it's a widely included header,
    that
results in ca 8 GB (2.5%) of extra code to compile during a full build.
```

This was due to implementing the `vector<bool>::reference` formatter specialization. This added several new includes to the `vector` header in `libc++`, like `<string>`, `<string_view>`, and `<locale>`.

Another motivation for this change is to avoid possible circular dependencies in headers. For example the `<vector>` header has a formatter specialization. Using `vector` in `<format>` causes a circular dependency. This circular dependency has caused an issue in `libc++`.

Note when using modules the inclusion size less relevant, but at the moment of writing module support for the standard library is not widely available on all platforms.

Does LEWG want to address this or should this be an LWG issue instead?

Does LEWG feel the translation unit size is something that should be addressed by LEWG?

§ 4.2 Complex

Both [P1636R2] and [P2197R0] have proposed this feature before. This implementation ideas align with [P2197R0].

The output format has two options:

- A stream-like output that is similar to the output of operator<<. This format still allows changing the output format of the floating-point values using the appropriate options of the *complex-format-spec*.
- A range-like output that matches more with the format used in [P2197R0].

The output is based on the range based formatters and uses the following *format-spec*:

complex-format-spec:

*range-fill-and-align*_{opt} *width*_{opt} *n*_{opt} *complex-type*_{opt} *complex-underlying-spec*_{opt}

complex-type:

s

r

complex-underlying-spec:

: *format-spec*

The *format-spec* of *complex-underlying-spec* is applied to both the real and the imaginary part of the complex number. This allows users to have control on how these values are formatted. When the *complex-type* is r there are some special cases:

- Since the sign is the separator between the real and imaginary part, the imaginary part should always have a sign. Otherwise it will be hard to determine where the real part ends and the imaginary part begins. Therefore the sign option for the imaginary defaults to + and the option - is ignored. This still allows to use the sign option.
- If the value is not infinity or NaN the value is suffixed with “i”, otherwise the value is suffixed with ” i”. The differences is just for readability purposes. The suffix is part of the formatted value and counts to its width.

The *complex-type* behaves like:

- s uses the output format similar to the operator<< output.

— The *complex-type* *r* uses the range based output format.

When no *complex-type* the range based output format is used.

These letters are not used in the current *std-format-spec* and alternative would be to use *o* or *s* for the stream-like output. The option *S* will also be used for the *error_condition* formatter where *o* (octal) and *s* (string) have a different meaning. For consistency the proposal uses *S* for all stream-like output formats.

Some examples of the output for formatting complex values

Value	Format	Output
<code>complex{0, 0}</code>	<code>{}</code>	<code>(0+0i)</code>
<code>complex{3, 4}</code>	<code>{}</code>	<code>(3+4i)</code>
<code>complex{-3, 4}</code>	<code>{}</code>	<code>(-3+4i)</code>
<code>complex{3, -4}</code>	<code>{}</code>	<code>(3-4i)</code>
<code>complex{nan, nan}</code>	<code>{}</code>	<code>(nan+nan i)</code>
<code>complex{-nan, -nan}</code>	<code>{}</code>	<code>(nan-nan i)</code>
<code>complex{inf, inf}</code>	<code>{}</code>	<code>(inf+inf i)</code>
<code>complex{-inf, -inf}</code>	<code>{}</code>	<code>(-inf-inf i)</code>
<code>complex{0, 0}</code>	<code>{:r}</code>	<code>(0+0i)</code>
<code>complex{0, 0}</code>	<code>{:nr}</code>	<code>0+0i</code>
<code>complex{0, 0}</code>	<code>{:S}</code>	<code>(0,0)</code>
<code>complex{0, 0}</code>	<code>{:nS}</code>	<code>0,0</code>
<code>complex{0, 0}</code>	<code>{::#}</code>	<code>(0.+0.i)</code>
<code>complex{1.345, 1.3}</code>	<code>{:##^18}</code>	<code>###(1.345+1.3i)###</code>
<code>complex{1.345, 1.3}</code>	<code>{::08}</code>	<code>(0001.345+0001.3i)</code>
<code>complex{1.345, 1.3}</code>	<code>{::+08}</code>	<code>(+001.345+0001.3i)</code>
<code>complex{1.345, 1.3}</code>	<code>{::E}</code>	<code>(1.345E+00+1.3E+00i)</code>
<code>complex{1.345, 1.3}</code>	<code>{::6E}</code>	<code>(1.345000E+00+1.300000E+00i)</code>

There are some open questions:

- Should the *i* suffix be considered as part of the width of the imaginary value? Currently it is, it could also be considered not to be and instead part of the total width of the output, which also happens for the brackets.
- Are there items of the wish list of [\[P2197R0\]](#) that should be implemented? The author feels we should not implement a different symbol for the imaginary unit at this time. Currently it's not possible to define the brackets or separator for range based formatters in the *format_spec*. The author has ideas how this can be done, but feels this should be a separate paper.

§ 4.3 Bitset

This type's formatting deviates from [P1636R2], the original proposal's output isn't bad, but it's very limited. Next to `to_string` the `bitset` has the members `to_ulong` and `to_ullong`, which allow an integral value. Another way to look at a `bitset` is a range of bits. Proposed are the following output formats:

- As a string. This uses the `bitset`'s `to_string()` member function. The contents are a representation of the value and not a specific string. This means the output can't be truncated with a precision option, nor does it have the debug option.
- As an integral. Since the value is not a real arithmetic value the value is always considered to be a positive value. Implementations can use the member functions `to_ulong()` or `to_ullong()` to extract the value. Implementations are not allowed to throw `overflow_error` when the value fits in an `unsigned long long`. The usage of `to_ulong()` is intended to be allowed as optimization when it is known the value will fit in an `unsigned long`, for example when using a `bitset<8>`.

When the value does not fit in an `unsigned long long`, it is implementation defined whether an implementation throws `overflow_error` or outputs the correct value. Implementations are allowed to change this behaviour depending on the display type. For example, throw an `overflow_error` for the decimal display type, but write values for the binary display type. The reason for this choice is that writing the arbitrary long output for bases 2, 8, and 16 is easier to implement than for base 10. Since the value is a set of bits, the base 10 value is likely less interesting. Note that the `string` or `range of bool` make it possible to always get all bits, regardless of the number of bits in the `bitset`. The provided implementation has an example where a binary display type never throws.

- As a range of `bool` values, where the underlying `bool` can be configured like the normal `bool` value.

The formatter will use the following `format-spec`:

bitset-format-spec:

*range-fill-and-align*_{opt} *#*_{opt} *0*_{opt} *width*_{opt} *L*_{opt} *n*_{opt} *bitset-type*_{opt} *bitset-underlying-spec*_{opt}

bitset-type: one of

`b B d o x X r s`

bitset-underlying-spec:

: format-spec

Most fields match the behaviour of the *std-format-spec* (22.14.2.2 [format.string.std]) or the *range-format-spec* (22.14.7.2 [format.range.formatter]).

The *n* option is only valid when the *bitset-type* option is *r*.

The *r* option of *bitset-type* selects the range-based output. This implies the underlying bits are outputted in the same way as the `bool` formatter.

The *bitset-underlying-spec* is only valid when the output is range-based.

The *format-spec* of the *bitset-underlying-spec* matches the *std-format-spec* of the `bool` type.

Some examples of the output for formatting `bitset` values

Value	Format	Output
<code>bitset<4>{0x5}</code>	<code>{}</code>	0101
<code>bitset<4>{0x5}</code>	<code>{:s}</code>	0101
<code>bitset<4>{0x5}</code>	<code>{:b}</code>	101
<code>bitset<4>{0x5}</code>	<code>{:#B}</code>	0B101
<code>bitset<4>{0x5}</code>	<code>{:r}</code>	[false, true, false, true]
<code>bitset<4>{0x5}</code>	<code>{:r:b}</code>	[0, 1, 0, 1]
<code>bitset<4>{0x5}</code>	<code>{:nr:b}</code>	0, 1, 0, 1

§ 4.3.1 Bitset reference

Like `vector<bool>` the returned type of the non-const member operator `[](size_t)` of a `bitset` is a proxy. The formatter for this proxy is modeled after the proxy of `vector<bool>`.

(Note that the provided `libc++` implementation tests for a `bitset<N>::const_reference`-like type too. This is due `libc++` returning a non-conforming proxy from the `const` member operator `[](size_t)` in some ABI versions. These tests also work on confirming implementations returning a `bool`.)

The *format-spec* for this type is identical to the *std-format-spec* of a `boolean`.

§ 4.4 Sub_match

This type's formatting matches [P1636R2], but the wording takes a different approach. It's likely `sub_match` uses a `contiguous_iterator`, forcing the range to be copied to a string and then copied to the output adds unneeded overhead. The wording gives implementations options to pick other types. For example a `basic_string_view` for a `contiguous_iterator` and a `basic_string` for a `non-contiguous_iterator`.

The *format-spec* for this type is identical to the *std-format-spec* of a string type.

§ 4.5 Diagnostics

§ 4.5.1 To specialize `wchar_t` or not to specialize `wchar_t`

All these formatters may use `error_category::name()`. This function returns a string and has no option to return a `wstring`. The same holds true for the message member function of `error_code` and `error_condition`.

A design consideration is whether or not to allow `wchar_t` formatter specializations for the diagnostics. In [P1636R2] the formatter specialization had the following effect:

```
basic_ostringstream<charT> o;  
o.imbue(locale::classic());  
o << ec;
```

If `charT` is a `wchar_t` it widens the `char` input to `wchar_t`.

The formatter specialization for `stacktrace_entry` in [P2693R1] took a different design approach, it doesn't support `wchar_t`. At the moment Zach Laine is working on improving Unicode support in C++. The paper [P2728R3] specifies a `utf_view` specialization

These should be added to the list of “the debug-enabled string type specializations” in [format.formatter.spec]. This allows `utf_view` to be used in `std::format()` and `std::print()`. The intention is that the formatter will transcode to UTF-8 if the formatter's `charT` is `char`, or to UTF-16 if the formatter's `charT` is `wchar_t` – if transcoding is necessary at all.

Here the conversion from `char` to `wchar_t` will be done using Unicode transcoding. For consistency, there should only be one way of transcoding in the format library. (There is already some transcoding in the library with *STATICALLY-WIDEN*, which is done by the compiler.) The C++26 cycle has just started. The author prefers to disallow `wchar_t` specializations for now. Once [P2728R3] is approved the author can write a small paper adding `wchar_t` support using Unicode transcoding.

§ 4.5.2 Error_code

This type's formatting deviates from [P1636R2]. The original proposal's output feels limited. It's based on the output of `operator<<`. Customizing the output of `operator<<` for “custom” types is non-trivial, but formatters don't have this limitation. Instead of limiting the output of the formatter, let's embrace it. Some of the limitations are that it's not possible to write the error's message which may contain useful information.

It could be argued that the decimal output may not be portable across platforms. For example `EOVERFLOW` has the value 75 on Linux and 132 on Windows. However this value is currently already used in `operator<<`, so this proposal follows the direction in this regard.

The formatter will use the following format-spec:

error-code-format-spec:

*fill-and-align*_{opt} *#*_{opt} *0*_{opt} *width*_{opt} *L*_{opt} *error-code-type*_{opt}

error-code-type:

error-code-type-value

error-code-type-message

error-code-type-ostream

error-code-type-value: one of

```

b B d o x X
error-code-type-message:
s
error-code-type-ostream:
S

```

Most fields match the behaviour of the *std-format-spec* (22.14.2.2 [\[format.string.std\]](#)).

When *error-code-type* is a *error-code-type-value* the value is formatted as an `int` obtained by calling the `value()` member function.

When *error-code-type* is a *error-code-type-message* the value is formatted as a `string` obtained by calling the `message()` member function.

When *error-code-type* is a *error-code-type-ostream* the value is formatted is the same way the output of operator<<. When this display type is used only the *fill-and-align* and *width* option may be present.

When the *error-code-type* is omitted it is formatted as-if the *error-code-type-ostream* has been specified.

Formatting the `make_error_code(errc::value_too_large);` may give the following results:

Format	Output
{}	Value too large for defined data type
{:s}	Value too large for defined data type
{:d}	75
{:S}	generic:75

§ 4.5.3 Error_condition

This type is similar to `error_code`, but it has no operator<<. It's unclear to the author what the historic reason for the difference is. This proposal proposes to add an `error_condition` formatter. This formatter behaves the same as the `error_code` formatter.

§ 4.5.4 Error_category

This formatter behaves the same as a formatter for a `string` type taking its value from the `name()` member function. Since these messages typical are short and truncating them may loose information the formatter specialization will not allow the precision option that is available for `string` types.

Formatting the `generic_category();` gives the following results:

Format	Output
{}	generic
{:s}	generic
{:.42}	// ill-formed, precision not allowed

§ 4.6 Byte

There are no previous proposals for this type. The type is intended to be used as a memory buffer containing bytes. With the presence of range-based formatting it seems useful to be able to format this memory buffer.

This type is defined in `cstddef`. It feels wrong to add a formatter to a `c` header. Instead this formatter specialization will be available in the `<format>` header.

Since the type is small it makes sense to directly store the byte in the `basic_format_arg` exposition only value variant. However this change may be problematic for implementers due to following requirement

22.14.8.3 [\[format.args\]/1](#)

An instance of `basic_format_args` provides access to formatting arguments.
 Implementations should optimize the representation of `basic_format_args` for a small number of formatting arguments.
 [Note 1: For example, by storing indices of type alternatives separately from values and packing the former. – end note]

Implementations have different methods for packing their types. Two of the implementations have added additional types to this “variant”:

- libc++ 128 bit integral types
- libstdc++ 128 bit integral types and extended floating-point types
- MSVC STL has no extensions

Instead of requiring implementers to add it to the variant it is unspecified whether the value is stored directly in the handle or in the variant. This difference is observable when users call `visit_format_arg`. Searching for this string on GitHub only finds usage of this function in {fmt}, LLVM, GCC, MSVC STL and the LWG issue repositories (or forks of them). So the function unlikely to be used in the wild. (Note [\[P2637R2\]](#) deprecated this function and added a `visit<R>` member function to `basic_format_arg` as its replacement.)

Not specifying this means it’s unspecified whether the `visit` functions can use this value. Requiring it to be always visitable basically requires storing in the value in the “variant”.

The formatter itself behaves like the formatter for `int` except that it does not allow the character display type. When formatting a byte as a char half of the values can not be represented in a char if char has a signed char as underlying type. For `wchar_t` all values can be represented. When a value can't be represented as a char the formatter has to throw an exception ([[tab:format.type.int](#)]). So instead of adding a possible unusable format option, don't add it at all. If users want to output a range of bytes as chars they have the option to use a `std::views::transform` and convert the byte to a char and then the char formatter will be used.

Since the type is always unsigned it might be worth considering to disallow a *sign* option. However the sign option can be used for unsigned integrals. Therefore the sign is allowed here too.

Some examples of the output for formatting byte values

Value		Format
byte{0}	{}	0
byte{42}	{:x}	2a
byte{255}	{:#X}	0X2A
byte{2}	{:b}	10
byte{2}	{:c}	// ill-formed, char output not allowed

§ 5 Impact on the standard

The proposal is a library only extension.

§ 6 Implementation experience

A libc++ [reference implementation](#) is available. The formatter byte has not been implemented. The author would like to get approval for the design direction before writing this formatter.

§ 7 Proposed wording

The wording is not entirely complete and needs polishing before LWG starts reviewing. For the same reason as the implementation, the formatter byte has not wording. The wording is based on [[N4944](#)].

§ 7.1 Feature test macro

Update the macro `__cpp_lib_formatters` to the date of adoption and make it available in the following additional headers `<bitset>`, `<complex>`, `<regex>`, `<system_error>`, `<format>`.

§ 7.2 Format's impact on translation unit size

22.14.6.3 [\[format.formatter.spec\]/2](#)

- Each header that declares the template formatter provides the following enabled specializations:
- + This header provides the following enabled specializations:

§ 7.3 Formatter complex

Add to 28.4.2 [\[complex.syn\]](#)

```
template<class T> complex<T> tan (const complex<T>&);
template<class T> complex<T> tanh (const complex<T>&);

+ // [complex.format], complex formatting
+ template<class T, class charT>
+ struct formatter<complex<T>, charT>;
+
+ // [complex.literals], complex literals
+ inline namespace literals {
+ inline namespace complex_literals {
```

Add a new section 26.4.? Formatting [\[complex.format\]](#):

```
namespace std {
    template<class T, class charT>
    class formatter<complex<T>, charT> {
        formatter<T, charT> underlying_;
        // exposition only
        basic_string_view<charT> separator_ = STATICALLY-WIDEN<charT>("");
        // exposition only
        basic_string_view<charT> opening-bracket_ = STATICALLY-WIDEN<charT>("
        ("); // exposition only
        basic_string_view<charT> closing-bracket_ = STATICALLY-WIDEN<charT>
        (")"); // exposition only

    public:
        constexpr void set_separator(basic_string_view<charT> sep) noexcept;
        constexpr void set_brackets(basic_string_view<charT> opening,
                                     basic_string_view<charT> closing)
            noexcept;

        template<class ParseContext>
        constexpr typename ParseContext::iterator
            parse(ParseContext& ctx);

        template<class FormatContext>
        typename FormatContext::iterator
            format(complex<T> value, FormatContext& ctx) const;
    };
}
```

1 template<class T, class charT> class formatter<complex<T>, charT>;

formatter<complex<T>, charT> interprets *format-spec* as a *complex-format-spec*. The syntax of format specifications is as follows:

```
complex-format-spec:
    range-fill-and-alignopt widthopt nopt complex-typeopt complex-underlying-
specopt
complex-type:
    s
    r
complex-underlying-spec:
    : format-spec
```

- 2 *range-fill-and-align* is interpreted the same way as a described in 22.14.7.2 [format.range.formatter].
- 3 *width* is interpreted the same way as a described in 22.14.2 [format.string].
- 4 *n* is interpreted the same way as a described in 22.14.7.2 [format.range.formatter].
- 5 the *complex-type* specifier changes the way a complex is formatted, with certain options only valid with certain argument types. The meaning of the various type options is as specified in Table xx.

Table xx: Meaning of complex-type options [tab:complex.format.type]

Option	Meaning
s	The output is compatible with the ostream output. Indicates the separator should be STATICALLY-WIDEN(“,”).
r	The output is a range of the real and imaginary part.
none	The same as r.

- 6 The *format-spec* in a *complex-underlying-spec*, if any, is interpreted as the *std-format-spec* for a floating-point type as described in 22.14.2 [format.string].

```
constexpr void set_separator(basic_string_view<charT> sep) noexcept;
```

- 7 *Effects:* Equivalent to: separator_ = sep;

```
constexpr void set_brackets(basic_string_view<charT> opening,
                           basic_string_view<charT> closing) noexcept;
```

- 8 *Effects:* Equivalent to:

```
opening-bracket_ = opening;
closing-bracket_ = closing;

template<class ParseContext>
constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```


9 *Effects:* Parses the format specifier as a *complex-format-spec* and stores the parsed specifiers in **this*. The values of *opening-bracket_*, *closing-bracket_*, and *separator_* are modified if and only if required by the *complex-type* or the *n* option, if present. The function *underlying_.parse()* is called with the *format-spec* of *complex-underlying-spec*.

10 *Returns:* An iterator past the end of the *complex-format-spec*.

```
template<class FormatContext>
    typename FormatContext::iterator
        format(complex<T> value, FormatContext& ctx) const;
```

11 *Effects:* Writes the following into *ctx.out()*, adjusted according to the *complex-format-spec*:

(11.1) - *opening-bracket_*,

(11.2) - *value.real()* via *underlying_*,

(11.3) - *separator_*,

(11.5) - *value.imag()* via *underlying_*. If *complex-type* is not S and *underlying_.parse()* was called without a *sign* option or a *sign* option -, adjust the output as-if *underlying_.parse()* was called with *sign* option +.

(11.5) - if *complex-type* is not S, if *value.imag()* is not infinity or NaN, write ``STATICALLY-WIDEN(" i")` else write `STATICALLY-WIDEN(" i")`, and

(11.6) - *closing-bracket_*.

```
[ Example xx:
string s0 = format("{} ", complex{0.0, 0.0});           // s0 has value:
    (0+0i)
string s1 = format("{::-", complex{0.0, 0.0});         // s1 has value:
    (0+0i)
string s2 = format("{:: }", complex{0.0, 0.0});        // s2 has value: ( 0
    0i)
string s3 = format("{::+", complex{0.0, 0.0});         // s3 has value:
    (+0+0i)
string s4 = format("{:S:}", complex{0.0, 0.0});        // s4 has value:
    (0,0)

double inf = numeric_limits<double>::infinity();
double nan = numeric_limits<double>::quiet_NaN();
string s5 = format("{::}", complex{inf, nan});         // s5 has value:
    (inf+nan i)
string s6 = format("{:S:}", complex{inf, nan});        // s6 has value:
    (inf,nan)
-- end example]
```

12 *Returns:* An iterator past the end of the output range.

§ 7.4 Formatters `bitset` and `bitset::reference`

Add to 22.9.1 [\[bitset.syn\]](#)

```
template<class charT, class traits, size_t N>
    basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const bitset<N>& x);

+ // [bitset.format], formatter specialization for bitset
+ template<size_t N, class charT> struct formatter<bitset<N>, charT>;
+
+ template<class T>
+     constexpr bool is-bitset-reference = see below;           //
+         exposition only
+
+ template<class T, class charT> requires is-bitset-reference<T>
+     struct formatter<T, charT>;
+
+ }
```

[3]{.pnum} The functions described in [\[template.bitset\]{.sref}](#) can report three kinds of errors, each associated with a distinct exception:

[3.1]{.pnum} an `_invalid_argument_error` is associated with exceptions of type ``invalid_argument`` ([\[invalid.argument\]{.sref}](#));

[3.2]{.pnum} an `_out-of-range_error` is associated with exceptions of type `out_of_range` ([\[out.of.range\]{.sref}](#));

[3.3]{.pnum} an `_overflow_error` is associated with exceptions of type `overflow_error` ([\[overflow.error\]{.sref}](#)).

```
+template<class T>
+     constexpr bool is-bitset-reference = see below;
+ [4]{.pnum} The expression is-bitset-reference<T> is true if T denotes
+     the type `template<size_t N> bitset<N>::reference`.
```

Add a new section 22.9.? Formatting [\[bitset.format\]](#):

```
namespace std {
    template<size_t N, class charT = char>
    class formatter<bitset<N>, charT {
        basic_string_view<charT> separator_ = STATICALLY-WIDEN<charT>("");
        // exposition only
        basic_string_view<charT> opening-bracket_ = STATICALLY-WIDEN<charT>("
        ("); // exposition only
        basic_string_view<charT> closing-bracket_ = STATICALLY-WIDEN<charT>("
        (")); // exposition only
        charT zero_ = CharT('0');
        // exposition only
        charT one_ = CharT('1');
        // exposition only

    public:
        constexpr void set_separator(basic_string_view<charT> sep) noexcept;
        constexpr void set_brackets(basic_string_view<charT> opening,
                                     basic_string_view<charT> closing)
            noexcept;
        constexpr void set_zero_one(charT zero, charT one) noexcept;
    }
```

```

template<class ParseContext>
constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);

template<class FormatContext>
typename FormatContext::iterator
    format(bitset<N> value, FormatContext& ctx) const;
};
}

```

1 `template<size_t N, class charT> class formatter<bitset<N>, charT>;`

`formatter<bitset<N>, charT>` interprets *format-spec* as a *bitset-format-spec*. The syntax of format specifications is as follows:

bitset-format-spec:

*range-fill-and-align*_{opt} *#*_{opt} *0*_{opt} *width*_{opt} *L*_{opt} *n*_{opt} *bitset-type*_{opt} *bitset-underlying-spec*_{opt}

bitset-type: one of

`b B d o x X r s`

bitset-underlying-spec:

`: format-spec`

2 *range-fill-and-align*, is interpreted the same way as a described in 22.14.7.2 [format.range.formatter].

3 *#*, *0*, *width*, and *L* are interpreted the same way as a described in 22.14.2 [format.string].

4 *n* is interpreted the same way as a described in 22.14.7.2 [format.range.formatter].

5 the *bitset-type* specifier changes the way a bitset is formatted, with certain options only valid with certain argument types. The meaning of the various type options is as specified in Table xx.

Table xx: Meaning of bitset-type options [tab:bitset.format.type]

Option	Meaning
b, B, d, o, x, X	Is formatted as if formatting the output of <code>bitset<N>::to_ulong()</code>. If the value can not be represented in an unsigned long long it is implementation defined whether the value is formatted as an integral or an <code>overflow_error</code> is thrown. Implementations may use different behavior for the different options. [<i>Note</i>: This allows to generate output for the <code>b</code> option and throwing for the <code>d</code> option. — <i>end note</i>]

Option	Meaning
r	The output is formatted as a range of boolean values.
s	Is formatted as if formatting the output of <code>bitset<N>::to_string(zero_, one_)</code> .
none	The same s.

- 6 The *format-spec* in a *bitset-underlying-spec*, if any, is interpreted as the *std-format-spec* for a `bool` type as described in 22.14.2 [format.string]. The *bitset-underlying-spec* is only valid when the *bitset-type* option is `r`.

```
template<class T>
constexpr bool is-bitset-reference = see below;
```

- 7 The variable template `is-bitset-reference` is true if `T` denotes the type `bitset<N>::reference` for some value of `N` and `bitset<N>` is not a program-defined specialization.

```
template<class T, class charT> requires is-bitset-reference<T>
struct formatter<T, charT> {
private:
    formatter<bool, charT> underlying_;    // exposition only

public:
    template <class ParseContext>
        constexpr typename ParseContext::iterator
            parse(ParseContext& ctx);

    template <class FormatContext>
        typename FormatContext::iterator
            format(const T& ref, FormatContext& ctx) const;
};
```

```
template <class ParseContext>
constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```

- 8 Effects: Equivalent to return `underlying_.parse(ctx)`;

```
template <class FormatContext>
typename FormatContext::iterator
    format(const T& ref, FormatContext& ctx) const;
```

- 9 Effects: Equivalent to return `underlying_.format(ref, ctx)`;

§ 7.5 Formatter `sub_match`

Add to 32.3 [re.syn]

```
template<class charT, class ST, class BiIter>
    basic_ostream<charT, ST>&
        operator<<(basic_ostream<charT, ST>& os, const sub_match<BiIter>&
            m);
```

```
+ // [re.submatch.format], formatting
+ template<class BiIter, class charT>
+ struct formatter<sub_match<BiIter>, charT>;
+
+ // [re.results], class template match_results
+ template<class BidirectionalIterator,
+         class Allocator = allocator<sub_match<BidirectionalIterator>>>
+     class match_results;
```

Add a new section 38.8.? Formatting [re.submatch.format]:

- 1 `formatter<sub_match<BiIter>, charT>` is a *debug-enabled* string type specialization (22.14.6.3 [format.formatter.spec]).
- 2 The formatter outputs the result of the `sub_match`'s `str()`.

§ 7.6 Header `system_error`

Add to 19.5.2 [system.error.syn]

```
// [syserr.compare], comparison operator functions
bool operator==(const error_code& lhs, const error_code& rhs) noexcept;
bool operator==(const error_code& lhs, const error_condition& rhs)
    noexcept;
bool operator==(const error_condition& lhs, const error_condition& rhs)
    noexcept;
strong_ordering operator<==(const error_code& lhs, const error_code&
    rhs) noexcept;
strong_ordering operator<==(const error_condition& lhs, const
    error_condition& rhs) noexcept;
+
+ // [syserr.format], formatter support
+ template<> struct formatter<error_category>;
+ template<> struct formatter<error_code>;
+ template<> struct formatter<error_condition>;
+
+ // [syserr.hash], hash support
+ template<class T> struct hash;
+ template<> struct hash<error_code>;
+ template<> struct hash<error_condition>;
```

Add a new section 19.5.? Formatting [syserr.format]:

- 1 For each of `error_code` and `error_condition`, the library provides the following formatter specialization where `error-code` is the name of the template.

```
template<> struct formatter<error-code>;
```

- 1 `formatter<error-code>` interprets *format-spec* as a *error-code-format-spec*. The syntax of format specifications is as follows:

error-code-format-spec:
*fill-and-align*_{opt} *#*_{opt} *0*_{opt} *width*_{opt} *L*_{opt} *error-code-type*_{opt}
error-code-type:
error-code-type-value
error-code-type-message
error-code-type-ostream
error-code-type-value: one of
b B d o x X
error-code-type-message:
s
error-code-type-ostream:
S

- 3 *fill-and-align*, *#*, *0*, *width*, and *L* are interpreted the same way as a described in 22.14.2 [format.string].
- 4 When *error-code-type* is a *error-code-type-value* the value is formatted as an `int` obtained by calling the `value()` member function.
- 5 When *error-code-type* is a *error-code-type-message* the value is formatted as a `string` obtained by calling the `message()` member function.
- 6 When *error-code-type* is a *error-code-type-ostream* the value is formatted is the same way the output of operator<<. When this display type is used only the *fill-and-align* and *width* option may be present.
- 7 When the *error-code-type* is omitted it is formatted as-if the *error-code-type-ostream* has been specified.

§ 7.7 Byte

TODO after the generic design is approved.

§ 8 References

[N4944] Thomas Köppe. 2023-03-22. Working Draft, Standard for Programming Language C++.

<https://wg21.link/n4944>

[P1636R2] Lars Gullik Bjønnes. 2019-10-06. Formatters for library types.

<https://wg21.link/p1636r2>

[P1684R5] Christian Trott, D. S. Hollman, Mark Hoemmen, Daniel Sunderland, Damien Lebrun-Grandie. 2023-05-19. `mdarray`: An Owning Multidimensional Array Analog of `mdspan`.

<https://wg21.link/p1684r5>

[P2197R0] Michael Tesch, Victor Zverovich. 2020-08-22. Formatting for `std::complex`.

<https://wg21.link/p2197r0>

[P2286R8] Barry Revzin. 2022-05-16. Formatting Ranges.

<https://wg21.link/p2286r8>

[P2585R0] Barry Revzin. 2022-05-15. Improving default container formatting.

<https://wg21.link/p2585r0>

[P2637R2] Barry Revzin. 2023-05-16. Member visit.

<https://wg21.link/p2637r2>

[P2693R1] Corentin Jabot, Victor Zverovich. 2023-02-09. Formatting thread::id and stacktrace.

<https://wg21.link/p2693r1>

[P2728R3] Zach Laine. 2023-05-10. Unicode in the Library, Part 1: UTF Transcoding.

<https://wg21.link/p2728r3>

[P2845R0] Victor Zverovich. 2023-05-07. Formatting of std::filesystem::path.

<https://wg21.link/p2845r0>